PERSONAL OS

Master Product & Technical Specification

Personal website + private workspace for identity, career, productivity, health, and writing.

Version 1.0 | Frozen planning baseline | September 2026

Document purpose

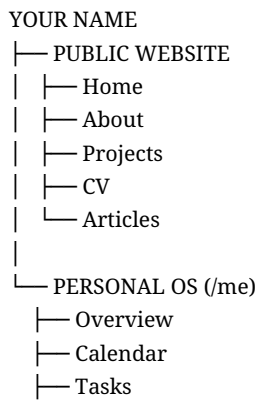
This document consolidates the product, UX, data model, technical architecture, security posture, infrastructure, and implementation roadmap agreed during planning. The PDF is the final planning artifact; future implementation changes should be treated as explicit deviations from this baseline.

1. Product Vision

Core concept: not "portfolio + admin dashboard", but a Personal OS where the public website represents the person and /me operates the person's day-to-day work and life.

- Public side answers: Who am I? What have I built? What do I know? What have I written? How can people contact me?
- Private side answers: What should I do today? Where are my applications? What is scheduled? How is my health trending? What am I thinking/writing?
- Single-user product with user-aware ownership and authorization.
- Public and private experiences share one visual language but differ in information density and interaction depth.

2. Product Scope and Domain Map



- ├── Career
- ├── Health
- ├── Journal
- ├── Articles
- └── Settings

V1 scope

Domain	Public	Private	V1
Profile / About	Yes	Owner edit	P0
Portfolio / Projects	Yes	Owner edit	P0
CV	Yes	Owner source data	P0
Articles	Published only	Draft/edit/publish	P0
Career tracking	No	Yes	P0
Calendar	No	Yes	P0
Tasks	No	Yes	P0
Health: weight	No	Yes	P0
Health: nutrition	No	Yes	P0/P1 within V1 scope
Journal	No	Yes	P0
Exercise	No	Yes	P1 / later V1

3. Information Architecture & Routes

Public routes

- /
- /about
- /projects
- /projects/[slug]
- /cv
- /articles
- /articles/[slug]
- /contact

Private routes

- /login
- /me
- /me/calendar
- /me/tasks
- /me/career
- /me/career/applications
- /me/career/applications/[id]
- /me/health
- /me/health/weight
- /me/health/nutrition
- /me/journal
- /me/journal/[id]
- /me/articles

/me/articles/[id]
/me/settings
/me/settings/profile
/me/settings/account
/me/settings/data

Private space uses /me and is intentionally not named /admin or /dashboard.

4. UI/UX Architecture

Public navigation

About | Projects | Articles | CV | /me

Private navigation

OVERVIEW

WORK
Calendar
Tasks
Career

LIFE
Health
Journal

CONTENT
Articles

SETTINGS
Profile / Account / Data

Primary pages

Page	Primary purpose	Main UI
/	Professional identity	Hero, selected work, experience snapshot, writing, contact
/projects/[slug]	Technical case study	Hero, sections, architecture, technologies, results, links
/cv	Recruitment asset	Profile, experience, education, skills, selected projects
/me	Daily command center	Today schedule, tasks, career, health, journal
/me/career/applications/[id]	Application workspace	Timeline, status, contacts, interviews, documents, notes
/me/calendar	Time planning	Day/week/month calendar, create/edit/move event
/me/health	Health overview	Weight, nutrition, exercise summaries and trends
/me/journal/[id]	Private writing	Focused editor, minimal chrome
/me/articles/[id]	Publication editor	Title, excerpt, editor, tags, cover, publish workflow

Interaction rules

- Quick capture -> dialog/drawer. Complex work -> dedicated page.
- Public content is editorial and whitespace-heavy; private content is denser and task-oriented.
- Mobile is not a shrunk desktop: tasks and health emphasize quick capture; calendar defaults to a single-day/agenda-oriented experience.
- Every page must have loading, empty, error, and success feedback states.

5. Final ERD / Database Blueprint

Entity groups

IDENTITY
users, profiles, social_links, experiences, experience_achievements, experience_skills, educations, skills, profile_skills

PORTFOLIO
projects, project_sections, project_skills, project_links, project_media, technologies, project_technologies

CONTENT
articles, tags, article_tags, journal_entries

CAREER
companies, applications, application_status_history, application_timeline_events, application_contacts, interviews, interview_contacts, application_documents

PRODUCTIVITY
tasks, calendar_events

HEALTH
weight_entries, foods, meals, meal_items, exercise_entries, health_goals

MEDIA
media

Core relationships

Parent	Child	Cardinality	Notes
users	profiles	1:1	Single public profile per owner
users	projects	1:N	Portfolio ownership
projects	skills	N:M	Via project_skills
users	articles	1:N	Published/draft/archived
articles	tags	N:M	Via article_tags
users	journal_entries	1:N	Default one entry per date in V1
companies	applications	1:N	Personal job-search records
applications	application_status_history	1:N	Status transitions
applications	application_timeline_events	1:N	Other timeline activities
applications	interviews	1:N	Interview scheduling

interviews	calendar_events	1:1	Interview can surface on calendar
users	tasks	1:N	Private task ownership
users	calendar_events	1:N	Internal calendar
users	weight_entries	1:N	Raw measurements
meals	meal_items	1:N	Nutrition consumption snapshot
foods	meal_items	1:N	Reusable food template
users	media	1:N	Owned files/metadata

Selected schema rules

- Primary keys use UUID.
- Time-aware fields use TIMESTAMPTZ; date-only concepts use DATE.
- Mutable tables use created_at / updated_at; historical event tables keep occurred_at or changed_at as appropriate.
- Public slugs are unique per owner where applicable.
- All private root entities carry user_id directly; child entities derive ownership through their parent where duplication is unnecessary.
- Calculated aggregates (weekly/monthly weight averages, nutrition totals, career counts) are derived from raw data rather than stored as authoritative rows.
- Meal items store nutritional snapshots so later edits to the food template do not rewrite history.
- CV is a projection from profile, experience, education, skills, and projects; it is not a required standalone database entity in V1.

6. Server Action & Query Contract

READ
UI / Server Component
-> Query
-> Service
-> Drizzle
-> PostgreSQL

WRITE
UI
-> Server Action
-> Auth
-> Zod validation
-> Authorization / ownership
-> Service / transaction
-> Drizzle
-> PostgreSQL
-> Revalidation

Representative actions

Domain	Read	Mutation
Profile	getProfile()	updateProfile(input)
Projects	getPublicProjects(), getProjectBySlug()	create/update/publish/reorder

Articles	getPublishedArticles()	create/update/publish/archive
Journal	getJournalEntries(range)	create/update/delete
Tasks	getTasks(filters)	create/update/complete/reopen/delete
Calendar	getCalendarEvents(range)	create/update/move/delete
Career	getApplications(filters)	create/update/changeStatus
Interviews	getInterviews(range)	create/update/cancel/complete
Weight	getWeightEntries(range), getWeightSummary()	create/update/delete
Nutrition	getMeals(date), getDailyNutritionSummary()	create/update/delete meal/items

Transaction boundaries

- Application status change: update current status + insert status history + timeline event atomically.
- Interview creation: create interview + calendar event + timeline record atomically.
- Application creation may create/reuse a company and create the first timeline/state record atomically.
- Meal creation with items should remain consistent as one logical write operation.

7. Technical Architecture & Stack

Layer	Decision	Rationale
Architecture	Modular monolith	One codebase; avoids microservice overhead
Framework	Next.js	Public SEO pages + private app in one system
Language	TypeScript	End-to-end type safety
UI	React + Tailwind CSS + shadcn/ui	Flexible personal visual system
Database	PostgreSQL	Relational data, JSONB, aggregation, constraints
ORM	Drizzle ORM	Typed schema and SQL-level control
Validation	Zod	Runtime input validation with TS inference
Auth	Auth provider abstraction; provider selected during setup	Keeps domain code provider-independent
Rich text	Tiptap-style structured JSON	Shared editor for project/article/journal
Charts	Recharts-class charting	Simple health/career visualization
Calendar	Calendar library behind a project wrapper	Avoids vendor lock-in in feature code
Storage	S3-compatible object storage	Images, CV, attachments
Testing	Vitest + Playwright	Unit/integration/E2E coverage
Package manager	pnpm	Simple TypeScript project management

8. Repository & Code Organization

```
src/
├── app/
│   ├── (public)/
│   ├── (auth)/
│   └── me/
├── components/
│   ├── ui/
│   ├── layout/
│   └── shared/
├── features/
│   ├── profile/
│   ├── portfolio/
│   ├── projects/
│   ├── articles/
│   ├── journal/
│   ├── career/
│   ├── tasks/
│   ├── calendar/
│   └── health/
├── server/
│   ├── auth/
│   ├── services/
│   └── repositories/
├── db/
│   ├── schema/
│   ├── migrations/
│   └── index.ts
├── lib/
└── types/
```

Dependency direction

UI -> Server Action / Query -> Service -> Data Access -> PostgreSQL

Components do not query the database directly. Business rules do not live in page components.

9. Security & Privacy Specification

Authentication / authorization

- All /me routes require an authenticated session.
- Every private mutation must verify current user ownership on the server.
- Knowing a UUID is never sufficient authorization.
- Public queries return only published/public-safe records.
- Draft articles, journal, career, tasks, calendar, and health data are private by default.

Sensitive-data policy

Data class	Examples	Rules
Public	Profile, projects, published articles	May be cached/indexed as appropriate
Private	Career, tasks, calendar, journal	No public query path
Highly private	Weight, nutrition, health history	Avoid third-party analytics/logging

		leakage

File/content security

- Use server-controlled storage keys and file validation; do not trust client filename/extension alone.
- Private documents use authorized/signed access rather than permanent public URLs.
- Rich text uses a controlled structured document model; do not render arbitrary HTML from untrusted input.
- Never log passwords, tokens, journal contents, health data, or private document contents.

10. Infrastructure & Deployment

```
GitHub
 -> CI
 -> Preview deployment
 -> main
 -> Production
    |
    +-> Next.js application
    +-> Managed PostgreSQL
    +-> S3-compatible storage
```

Environments

- Local: development-only data/configuration.
- Preview: isolated environment for pull requests and QA.
- Production: real personal data and custom domain.

Operational controls

- Managed database with automated backups; never use production as the development database.
- Versioned migrations in Git; destructive schema changes require extra review.
- Secrets live only in environment/secret management systems.
- Database and object-storage recovery must both be considered; a DB backup alone is insufficient.
- V1 target is recoverable service, not enterprise zero-downtime infrastructure.

11. Testing Strategy

Layer	Focus	Examples
Unit	Pure business rules	Weight averages, nutrition totals, status rules, date handling
Integration	Actions + DB + auth	Create application, change status, create interview, add meal
E2E	Critical user journeys	Login, portfolio publish, career flow, health logging, journal write
Security	Ownership/privacy	User cannot read another user private record even with known ID

Definition of Done

- Functional behavior works for the intended user flow.

- Validation, authorization, and ownership checks are covered.
- Loading, empty, error, and success states exist.
- Business-critical logic has automated tests.
- Mobile behavior is verified for the feature.

12. Implementation Roadmap

Phase	Goal	Key deliverables
0	Foundation	Next.js, TypeScript, Tailwind, shadcn, lint, CI, deployment skeleton
1	Database + Auth	PostgreSQL, Drizzle, migrations, authentication, protected /me
2	Public Website	Home, About, Projects, Project detail, CV, Articles
3	Personal OS Shell	/me overview, layout, navigation, shared states
4	Tasks + Calendar	Task CRUD, event CRUD, calendar view and interactions
5	Career	Applications, status history, timeline, contacts, interviews, documents
6	Health	Weight, averages/trends, meals, nutrition summaries
7	Journal + Articles	Rich editor, journal, article draft/publish workflow
8	Media + Integration	Storage flow, command actions, cross-domain polish
9	Hardening	Security audit, accessibility, performance, SEO, tests, backup verification
10	Production V1	Smoke tests, recovery checks, final deployment

13. Intentionally Deferred Features

- Google Calendar synchronization
- Recurring task engine and advanced recurring event exceptions
- AI assistant / AI writing / AI career insights
- Multi-user collaboration and role-based access control
- Global search/command palette beyond basic navigation actions
- Advanced health metrics such as sleep, body fat, blood pressure, wearable integrations
- Advanced application scraping, email sync, LinkedIn sync, recruiter integrations
- Article revision history/versioning
- Dedicated analytics warehouse or materialized analytics tables unless performance requires them
- Contact form message storage and notification platform

14. Engineering & Product Principles

- Simple now, extensible later: do not build infrastructure for hypothetical scale.
- Public/private separation is a security boundary, not just a navigation concept.
- Raw data is source of truth; derived statistics are computed.
- Historical records should remain historically stable when master/reference data changes.
- Quick capture matters: task, weight, meal, journal, and application entry should be fast.
- The private UI should feel like the same person's website, not a generic admin template.
- Cross-domain integrations should represent real semantic relationships, not arbitrary foreign keys.
- Prefer explicit relationships over polymorphic database tricks when foreign-key integrity matters.

15. Final Release Checklist

Area	Check
Product	V1 scope matches the frozen blueprint
Database	All migrations reproducible and indexes verified
Security	Auth, ownership, private routes, upload rules verified
UX	Responsive, keyboard/focus, loading/empty/error states
SEO	Public pages have metadata, sitemap, robots policy
Performance	Images, bundles, database queries reviewed
Testing	Critical flows pass unit/integration/E2E tests
Operations	Backups configured and restore path tested
Storage	Public/private file access verified
Deployment	Production smoke test passes

Appendix A - Core Data-to-UI Flow

Public project:
Project data -> public query -> server-rendered project page

Career interview:
Application -> Interview -> Calendar Event -> /me/calendar

Health:
Weight entries -> aggregation -> weekly/monthly summary -> chart
Meal items -> nutrition aggregation -> daily summary -> health overview

Journal:
Date -> journal entry -> structured JSON content -> private editor

Appendix B - V1 Non-Goals

- This is not a general CMS for multiple authors.
- This is not a full ATS for hiring teams.
- This is not a full fitness or medical application.

- This is not a replacement for Google Calendar in V1.
- This is not a social publishing platform.
- This is not a microservices demonstration project.

Appendix C - Freeze Note

This document represents the agreed V1 planning baseline. Changes discovered during implementation should be documented as deliberate deviations, especially changes to data ownership, public/private boundaries, or cross-domain relationships.

END OF V1 MASTER SPECIFICATION